PopConly

Bitácora de fixes

Qué se rompió, por qué, y cómo se arregló.
Octubre 2026 · popconly.com

1. Notificaciones persistentes (estilo plataformas grandes)

Problema

El panel ocupaba toda la pantalla; al abrir la campana todo se marcaba leído y al recargar “todo cambiaba”; el punto rojo no se apagaba; no había forma de quitar una notificación.

Causa

Sin altura máxima ni scroll; auto-leído al abrir; el aviso de “leída” se cancelaba al navegar a /watch antes de llegar al servidor; sin endpoint de descarte.

Arreglo

Panel con altura máxima y scroll; leído solo al hacer clic en cada notificación; POST con keepalive para que sobreviva a la navegación; botón X por notificación (visible al hover, siempre visible en celular) que la elimina en Redis por usuario+perfil y la registra como descartada para que el algoritmo jamás la regenere.

Verificación

tsc limpio, 17/17 tests, desplegado y operando en producción.

2. Carátulas que a veces no se veían al recargar (Series)

Problema

Al recargar en Series, algunas carátulas quedaban en gris de forma intermitente.

Causa

Doble: los héroes pedían la imagen original completa sin caché (pesada y lenta, compitiendo con las 11+ miniaturas); y si el navegador ya tenía la imagen en caché y la pintaba antes de hidratar, el aviso de carga se perdía y la carátula quedaba invisible. El servidor estaba sano (10/10 miniaturas en ~0.4 s).

Arreglo

Todos los héroes y fondos usan miniatura cacheada (w=1280/640); /api/stream marca toda imagen como immutable; BackdropCard usa miniatura; CoverImg detecta por ref si la imagen ya está completa y se muestra de inmediato.

Verificación

Headers Cache-Control verificados en producción; tsc + tests en verde.

3. Iconos rotos en conexiones lentas (texto “check” visible)

Problema

En el onboarding de pago se leían las palabras check / check_circle / chevron_right en vez de los iconos; la página parecía rota.

Causa

La fuente de iconos pesaba 4 MB (set completo). En conexiones lentas no cargaba antes del control de 6 s y la app mostraba el respaldo en texto.

Arreglo

Subset a los 81 iconos que usa la app: 70 KB (57× menos). Se verificó glifo por glifo (normal y relleno, ejes FILL/pesos intactos). Quedó scripts/subset-symbols.py para regenerarla si se agrega un icono.

Verificación

Producción sirve la fuente de 70 KB en <1 s; tsc + tests en verde.

4. Icono de la pestaña (favicon)

Problema

La pestaña mostraba el icono genérico del navegador.

Causa

No había favicon del proyecto.

Arreglo

Desde el PNG con fondo transparente se generó el juego completo (favicon.ico multi-tamaño, icon-32, apple-touch-icon 180, master 512, con el punto naranja) y se conectó en el metadata del layout.

Verificación

Los 3 archivos responden 200 y el HTML emite los link rel=icon.

5. Idioma global ES/EN con persistencia por usuario

Problema

Cada componente guardaba el idioma por su cuenta: cambiarlo en el pie solo cambiaba el pie.

Causa

Estado local disperso sin proveedor global ni guardado en servidor.

Arreglo

Proveedor global de idioma; columna User.lang con default es y endpoint /api/me/settings; el cambio aplica a toda la app al instante y persiste por usuario entre sesiones y dispositivos (en landing sin sesión vive en el navegador). Los títulos nunca se traducen (son datos).

Verificación

Migración aplicada en producción (columna lang existe); tsc + tests en verde.

6. Escape no limpiaba la búsqueda

Problema

Pulsar Escape colapsaba el buscador pero dejaba el texto y los resultados.

Causa

El handler solo cerraba el campo sin limpiar la query.

Arreglo

Escape ahora limpia la query y restaura la pestaña como estaba.

Verificación

tsc + tests en verde, desplegado.

Cada fix se verificó con tsc + 17/17 tests y deploy con healthcheck en producción antes de darse por cerrado.